

Lembar Materi & Praktikum: Integrasi Microservices dengan gRPC dan Redis Caching pada Gin Backend

Durasi: 3 Jam (180 Menit) **Level:** Advanced (Lanjut)

Praktikum ini melengkapi materi teori pada [19. grpc-microservice.md](#). Anda akan dipandu untuk mengimplementasikan integrasi arsitektur layanan terdistribusi (**Microservices**), protokol komunikasi biner berkecepatan tinggi (**gRPC**), serta caching in-memory (**Redis**) pada proyek **Sistem Tiket Konser** (`go-tiket-konser`) dengan framework **Gin**. Kita akan belajar memisahkan fungsionalitas Monolith ke dalam microservices mandiri, menyusun kueri gRPC Server-Client, dan mempercepat respons HTTP API Gateway melalui penerapan pola **Cache-Aside (Read-Through Caching)**.

Sesi 1: Teori Arsitektur Microservices, HTTP/2, dan gRPC (45 Menit)

1. Monolith vs Microservices

- **Arsitektur Monolith:** Seluruh modul aplikasi (seperti Pengguna, Konser, dan Pembayaran) digabungkan dalam satu basis kode dan satu proses server tunggal.
 - *Kelemahan:* Jika modul Pembayaran mengalami kegagalan fatal (*out of memory* atau crash), seluruh server akan mati sehingga modul Pengguna dan Konser ikut terganggu. Skalabilitas juga tidak efisien karena kita harus menduplikasi seluruh aplikasi besar walaupun hanya modul Pembayaran yang menerima lonjakan traffic.
- **Arsitektur Microservices:** Memecah satu aplikasi besar menjadi banyak service kecil independen yang berjalan di prosesnya sendiri dan memiliki database terisolasi.
 - *Kelebihan:* Skalabilitas parsial (hanya men-scale service yang sibuk), isolasi kesalahan (jika service pembayaran crash, katalog konser tetap bisa diakses), dan kemudahan pembagian kerja tim.

2. Mengapa Memilih gRPC untuk Komunikasi Antar-Layanan (Backend-ke-Backend)?

Komunikasi eksternal (dari Aplikasi HP/Web ke Backend Gateway) sangat ideal menggunakan **JSON REST API** karena kemudahan integrasi. Namun, komunikasi internal (dari Service Gateway ke Service

Pembayaran/Konser) membutuhkan performa tinggi.

- **Masalah JSON:** Payload berbasis teks biasa (banyak spasi, tanda kutip, string) membutuhkan ukuran transfer yang besar dan beban CPU yang tinggi untuk proses serialisasi/deserialisasi JSON.
- **Solusi gRPC:** Mengirimkan data dalam format **Biner** yang sangat padat menggunakan **Protocol Buffers (Protobuf)** melalui jaringan **HTTP/2**. gRPC mendukung multiplexing (mengirim banyak request dalam satu koneksi TCP secara simultan), kompresi header, dan komunikasi dua arah (*bidirectional streaming*).

Sesi 2: Teori Redis Caching & Pola Cache-Aside (45 Menit)

1. Redis Sebagai In-Memory Database

Database relasional seperti PostgreSQL menyimpan data pada penyimpanan fisik (SSD/Harddisk). Kueri database relasional membutuhkan waktu sekitar 50ms s.d 200ms karena kueri I/O disk dan pemrosesan join tabel yang rumit.

Redis menyimpan data sepenuhnya di dalam **RAM**. Waktu baca-tulis Redis sangat cepat (sekitar 1ms s.d 5ms). Redis sangat cocok digunakan untuk menyimpan data yang sering dibaca tetapi jarang berubah (seperti katalog konser atau informasi kategori tiket).

2. Algoritma Caching: Cache-Aside Pattern

Pola caching yang paling umum digunakan di industri adalah **Cache-Aside (Read-Through)** dengan alur kerja berikut:

```
graph TD
    A[Klien kirim GET /concerts/:id] --> B{Cari di Cache Redis}
    B -- Ada (Cache HIT) --> C[Kembalikan Data Instan ke Klien]
    B -- Tidak Ada (Cache MISS) --> D[Kueri ke Database / gRPC Service]
    D --> E[Simpan Data Baru ke Redis dengan TTL]
    E --> F[Kembalikan Data ke Klien]
```

- **Cache HIT:** Data ditemukan di Redis. Backend langsung mengembalikan data tersebut tanpa menyentuh database. Waktu respons sangat cepat.
 - **Cache MISS:** Data tidak ada di Redis (atau sudah kedaluwarsa). Backend terpaksa melakukan kueri ke database/gRPC service, menyimpan hasilnya ke Redis dengan masa aktif (**TTL - Time To Live**), lalu mengirim respons ke klien.
 - **Time to Live (TTL):** Batas waktu kedaluwarsa data di Redis (misalnya 5 menit) untuk menjamin data di cache otomatis terhapus dan diperbarui jika ada perubahan di database utama.
-

Sesi 3: Laboratorium Praktikum Mandiri – Definisi Protobuf & gRPC Service (45 Menit)

Pelajari langkah-langkah pendefinisian kontrak gRPC dan pembuatan server layanan katalog konser di bawah ini.

1. Menulis Kontrak Protocol Buffers: `pb/concert.proto`

Buat folder baru bernama `pb/` di root proyek Anda, lalu buat file `concert.proto` untuk mendefinisikan interface fungsi katalog konser:

```
syntax = "proto3";

package pb;

option go_package = "./pb";

// Mendefinisikan struktur data Concert
message ConcertDetail {
    int32 id = 1;
    string title = 2;
    string description = 3;
    string date = 4;
    string venue = 5;
    string status = 6;
}

// Request masukan berupa ID konser
message ConcertRequest {
    int32 id = 1;
}

// Response keluaran berupa detail data konser
message ConcertResponse {
    ConcertDetail concert = 1;
}

// Definisikan Service Katalog Konser
service ConcertCatalog {
    rpc GetConcertDetail (ConcertRequest) returns (ConcertResponse);
}
```

Jalankan instalasi generator dan kompilasi proto dari terminal:

```
# 1. Install generator plugin
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```

```
# 2. Compile berkas proto menjadi file Golang otomatis
protoc --go_out=. --go-grpc_out=. pb/concert.proto
```

Hasil kompilasi akan otomatis menghasilkan berkas `pb/concert.pb.go` dan `pb/concert_grpc.pb.go`.

2. Membuat gRPC Server: `grpc_server/main.go`

Buat folder baru `grpc_server/` dan susun implementasi server gRPC independen yang membaca database GORM:

```
package main

import (
    "context"
    "log"
    "net"

    "go-tiket-konser/models"
    "go-tiket-konser/pb"

    "google.golang.org/grpc"
    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"
)

type concertServer struct {
    pb.UnimplementedConcertCatalogServer
    db *gorm.DB
}

// GetConcertDetail mengimplementasikan logika RPC untuk mengambil data dari database
relasional
func (s *concertServer) GetConcertDetail(ctx context.Context, req *pb.ConcertRequest)
(*pb.ConcertResponse, error) {
    var concert models.Concert

    // Kueri database utama
    err := s.db.First(&concert, req.GetId()).Error
    if err != nil {
        if err == gorm.ErrRecordNotFound {
            return nil, status.Errorf(codes.NotFound, "konser dengan ID %d
tidak ditemukan", req.GetId())
        }
        return nil, status.Errorf(codes.Internal, "database error: %v", err)
    }
}
```

```

// Bentuk response sesuai format Protobuf
return &pb.ConcertResponse{
    Concert: &pb.ConcertDetail{
        Id:          int32(concert.ID),
        Title:       concert.Title,
        Description: concert.Description,
        Date:       concert.Date.Format("2006-01-02"),
        Venue:      concert.Venue,
        Status:     concert.Status,
    },
}, nil
}

func main() {
    // Koneksi ke Database khusus gRPC server
    dsn := "postgres://postgres:secret45@localhost:5434/eticketdb?sslmode=disable"
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})
    if err != nil {
        log.Fatalf("gRPC DB gagal konek: %v", err)
    }

    // Buka socket TCP di port 50052
    lis, err := net.Listen("tcp", ":50052")
    if err != nil {
        log.Fatalf("Gagal mendengarkan port: %v", err)
    }

    s := grpc.NewServer()
    pb.RegisterConcertCatalogServer(s, &concertServer{db: db})

    log.Println("gRPC Catalog Server berjalan di port 50052...")
    if err := s.Serve(lis); err != nil {
        log.Fatalf("Gagal menyalakan gRPC server: %v", err)
    }
}

```

Sesi 4: Laboratorium Praktikum Mandiri – Integrasi gRPC Client & Redis Cache pada Gin (45 Menit)

Kita akan memodifikasi API Gateway berbasis Gin agar bertindak sebagai gRPC client yang memanggil gRPC Server, sekaligus mengamankan traffic kueri menggunakan Redis Caching.

1. Inisialisasi Klien Redis pada Aplikasi Gin: [config/redis.go](https://github.com/redis/go-redis)

Pertama, instalasi pustaka redis klien:

```
go get -u github.com/redis/go-redis/v9
```

Buat file baru `redis.go` di bawah folder `config/` :

```
package config

import (
    "context"
    "log"

    "github.com/redis/go-redis/v9"
)

var RedisClient *redis.Client

// InitRedis menghubungkan koneksi ke server Redis lokal
func InitRedis() {
    RedisClient = redis.NewClient(&redis.Options{
        Addr:      "localhost:6379", // Address default Redis
        Password: "",               // Tanpa password secara default
        DB:        0,               // Database index default
    })

    // Test Ping koneksi
    ctx := context.Background()
    _, err := RedisClient.Ping(ctx).Result()
    if err != nil {
        log.Fatal("Gagal menghubungkan ke Redis: ", err)
    }

    log.Println("Koneksi Redis sukses terhubung")
}
```

2. Modifikasi Handler API Gateway Gin: [handler/concert.go](#)

Modifikasi method `GetConcertByID` pada Gin controller Anda agar menerapkan pola **Cache-Aside** dan memicu pemanggilan gRPC Client jika terjadi cache miss:

```
package handler

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "strconv"
    "time"
```

```

        "go-tiket-konser/config"
        "go-tiket-konser/dto"
        "go-tiket-konser/pb"

        "github.com/gin-gonic/gin"
        "github.com/redis/go-redis/v9"
        "google.golang.org/grpc"
        "google.golang.org/grpc/credentials/insecure"
    )

    // Ambil detail konser menggunakan alur Cache-Aside dan komunikasi gRPC
    func (h *ConcertHandler) GetConcertByID(c *gin.Context) {
        idStr := c.Param("id")
        id, err := strconv.Atoi(idStr)
        if err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error": "ID tidak valid"})
            return
        }

        ctx := context.Background()
        cacheKey := fmt.Sprintf("concert:%d", id)

        // LOGIKA STEP 1: Cek Cache Redis (Cache-Aside Pattern)
        val, err := config.RedisClient.Get(ctx, cacheKey).Result()
        if err == nil {
            // CACHE HIT: Data ditemukan di Redis, langsung unmarshal ke struct
            Response
            var cachedConcert dto.ConcertResponse
            _ = json.Unmarshal([]byte(val), &cachedConcert)

            c.JSON(http.StatusOK, dto.WebResponse{
                Success: true,
                Message: "Berhasil menampilkan detail konser (Sumber: Cache
                Redis)",
                Data: cachedConcert,
            })
            return
        }

        // LOGIKA STEP 2: CACHE MISS - Hubungi gRPC Microservice Server di port 50052
        if err == redis.Nil {
            // Hubungkan socket ke gRPC Server
            conn, errGrpc := grpc.Dial("localhost:50052",
            grpc.WithTransportCredentials(insecure.NewCredentials()))
            if errGrpc != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error": "Gagal
                menghubungi gRPC microservice"})
                return
            }
            defer conn.Close()

            grpcClient := pb.NewConcertCatalogClient(conn)

```

```

        // Panggil fungsi RPC GetConcertDetail
        req := &pb.ConcertRequest{Id: int32(id)}
        res, errCall := grpcClient.GetConcertDetail(ctx, req)
        if errCall != nil {
            c.JSON(http.StatusNotFound, dto.WebResponse{
                Success: false,
                Message: "Konser tidak ditemukan di microservice",
            })
            return
        }

        // Konversi data gRPC response ke DTO standard
        concertResponse := dto.ConcertResponse{
            ID:          int(res.GetConcert().GetId()),
            Title:        res.GetConcert().GetTitle(),
            Description:  res.GetConcert().GetDescription(),
            Date:        res.GetConcert().GetDate(),
            Venue:      res.GetConcert().GetVenue(),
            Status:     res.GetConcert().GetStatus(),
        }

        // LOGIKA STEP 3: Simpan data ke Redis Cache dengan TTL 5 Menit agar
        kueri berikutnya menjadi HIT
        marshalled, _ := json.Marshal(concertResponse)
        _ = config.RedisClient.Set(ctx, cacheKey, marshalled,
            5*time.Minute).Err()

        c.JSON(http.StatusOK, dto.WebResponse{
            Success: true,
            Message: "Berhasil menampilkan detail konser (Sumber: gRPC
Microservice)",
            Data: concertResponse,
        })
        return
    }

    c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
}

```

Jalankan inisialisasi Redis di file `main.go` Anda:

```

func main() {
    // ... (inisialisasi db & logging) ...

    // Inisialisasi koneksi Redis
    config.InitRedis()

    // ... (setup router & start) ...
}

```


Sesi 5: Tantangan Praktikum Mandiri (Tugas Mandiri)

Selesaikan tantangan berikut untuk melatih konsistensi sinkronisasi data cache (*cache invalidation*) dan perancangan endpoint berkinerja tinggi.

Tantangan 1: Cache Invalidation Otomatis Saat Update/Delete Konser (Cache Eviction)

Masalah utama dari pola caching adalah data kedaluwarsa (*stale data*). Jika data konser dengan ID 1 diubah namanya di database lewat API `PUT /api/v1/concerts/1`, data di Redis Cache akan tetap menyimpan nama lama sampai durasi TTL 5 menit habis.

- **Tugas:**

1. Buka handler pemrosesan update dan delete konser Anda di `handler/concert.go`.
2. Begitu proses update atau delete database berhasil diproses, lakukan **Cache Eviction** (penghapusan kunci cache) secara terprogram dari Redis:

```
cacheKey := fmt.Sprintf("concert:%d", id)
config.RedisClient.Del(ctx, cacheKey)
```

3. Dengan cara ini, request pembacaan berikutnya terpaksa mengalami Cache Miss dan menarik data baru ter-update dari database.
-

Tantangan 2: gRPC Service untuk Validasi Sisa Kuota Tiket (Ticket Inventory Service)

Saat terjadi peristiwa penjualan tiket secara rebutan (*war tiket*), membiarkan ribuan request per detik mengakses database relasional langsung untuk memeriksa ketersediaan kuota tiket akan membuat server crash.

- **Tugas:**

1. Rancang berkas `.proto` tambahan untuk mendefinisikan RPC service `TicketInventory` yang memuat fungsi check kuota `CheckQuota(TicketCategoryRequest) returns (QuotaResponse)`.
2. Implementasikan gRPC Server khusus kuota yang menyimpan status sisa kuota tiket konser di Redis dengan struktur key `ticket:quota:<category_id>` dan TTL sangat pendek (misal: 10 detik).
3. Gunakan gRPC client ini di dalam middleware atau flow pemesanan tiket Anda sebelum database transactions dijalankan.